



ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΥΠΡΟΥ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Ομαδική Εργασία – Δεντρικές Δομές Δεδομένων

Μέρος 1^ο

Εφαρμογή Πρότασης Εναλλακτικών Λέξεων με Χρήση Δομής Trie

1. Υλοποίηση Λεξικού με Δομή Trie και Πίνακα Κατακερματισμού Robin Hood

Robin Hood Hashing

- Αποτελεί μορφή κατακερματισμού **ανοικτής διεύθυνσης** η οποία προσπαθεί να εξισορροπήσει τους χρόνους αναζήτησης των στοιχείων χρησιμοποιώντας μια πιο **δίκαιη στρατηγική χειρισμού των συγκρούσεων**.

– μειώνει τον μέσο όρο του "προβλεπτικού μήκους" αναζήτησης, περιορίζοντας το πόσο μακριά από την αρχική του θέση έχει μετακινηθεί το κάθε στοιχείο λόγω συγκρούσεων.

- Είναι αποδοτική για περιπτώσεις που θέλουμε να ελαχιστοποιήσουμε το χρόνο αναζήτησης στοιχείων σε ένα γεμάτο πίνακα κατακερματισμού.

– Αποδεικνύεται ότι το αναμενόμενο κόστος αναζήτησης χειρότερης περίπτωσης είναι φραγμένο από **$\Theta(\log \log n)$** .

```
public class RobinHoodTrie implements UsefulNumbers {  
  
    private RobinTrieNode root; // Root node of the RobinHoodTrie, starts with an empty node
```

```
    public class RobinTrieNode implements UsefulNumbers {  
  
        /* A reference to the RobinHoodHashing object associated with this TrieNode */  
        RobinHoodHashing robinHood;  
        int wordLength; // Stores the length of the word if it ends at this node  
        int importance; // The importance value associated with the word
```

```
    public class RobinHoodHashing implements UsefulNumbers {  
  
        private Element[] table; // The hash table holding the elements  
        private int capacity; // The capacity of the table  
        private int size; // The number of elements currently in the table  
        private int maxProbeLength; // The maximum probe length encountered
```

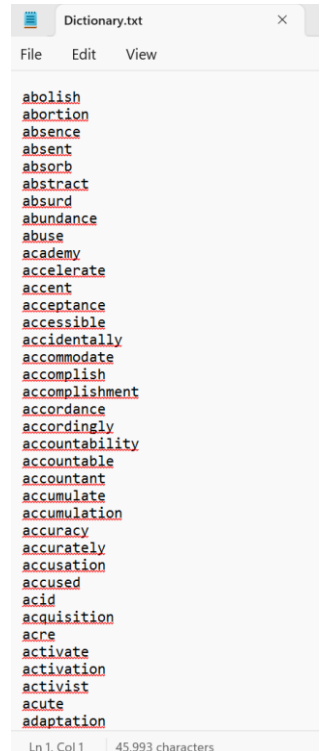
```
    public class Element implements UsefulNumbers {  
  
        private char key; // The character key of the element  
        private int probeLength; // The probe length from the initial hash position  
        private RobinTrieNode trieNode; // The RobinTrieNode that this Element points to
```

2. Ανάγνωση και Επεξεργασία Αρχείων Εισόδου

Το πρόγραμμά μας δέχεται σαν όρισμα (από τη γραμμή εντολής) δύο αρχεία:

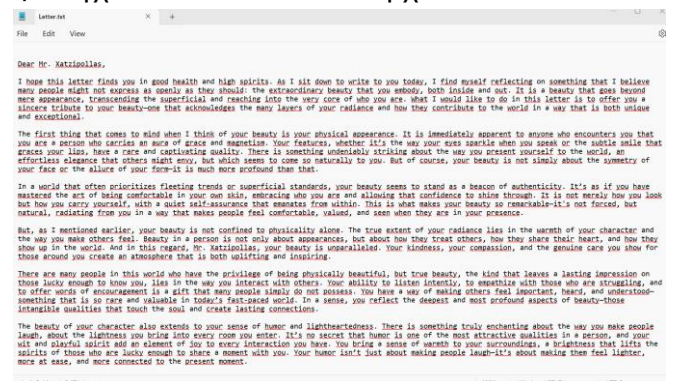
- **Αρχείο Λεξικού:** Περιέχει μια λέξη ανά γραμμή, χωρίς σημεία στίξης ή άλλους χαρακτήρες. Κάθε λέξη του αρχείου θα εισάγεται στο Trie.

```
public static void insertDictionaryRobinHood(String fileNameInput , RobinHoodTrie trie) {  
    try (BufferedReader br = new BufferedReader(new FileReader(fileNameInput))) {  
        String lineInput;  
        /* Read file line by line */  
        while ((lineInput = br.readLine()) != null) {  
            /*  
             * Splits the line from the search file into words using a regular expression.  
             * The regular expression "\\W+" matches one or more non-word characters  
             * (i.e., characters that are not letters, digits, or underscores),  
             * which means the line will be split into words wherever such characters are found.  
             * This effectively separates the line into individual words, removing any punctuation or special characters.  
             */  
            String[] words = lineInput.split("\\W+");  
            /* Insert each word into the RobinHoodTrie */  
            for(String word : words) {  
                word = word.toLowerCase(); // Convert the word to lowerCase before inserting  
                trie.insert(word); // Insert the word into the RobinHoodTrie  
            }  
        }  
    } catch (IOException e) {  
        /* Handle any exceptions that occur while reading the file */  
        System.err.println("Error reading the file: " + e.getMessage());  
    }  
}
```



- **Αρχείο Σημαντικότητας:** Περιέχει κείμενο (με τη γενική μορφή ενός αρχείου κειμένου) από το οποίο θα αντληθεί η συχνότητα εμφάνισης των λέξεων από το λεξικό. Κάθε φορά που μια λέξη του λεξικού εντοπίζεται στο κείμενο, η τιμή της σημαντικότητας της λέξης αυξάνεται. Οι λέξεις του αρχείου θα πρέπει να διαχωρίζονται από τα σημεία στίξης πριν αυτές ελεγχθούν κατά πόσο υπάρχουν στο λεξικό.

```
public static void searchDictionaryRobinHood(String fileNameSearch , RobinHoodTrie trie) {  
    try (BufferedReader br = new BufferedReader(new FileReader(fileNameSearch))) {  
        String lineSearch;  
        /* Read file line by line */  
        while ((lineSearch = br.readLine()) != null) {  
            /*  
             * Splits the line from the search file into words using a regular expression.  
             * The regular expression "\\W+" matches one or more non-word characters  
             * (i.e., characters that are not letters, digits, or underscores),  
             * which means the line will be split into words wherever such characters are found.  
             * This effectively separates the line into individual words, removing any punctuation or special characters.  
             */  
            String[] words = lineSearch.split("\\W+");  
            /* Search for each word in the RobinHoodTrie */  
            for(String word : words) {  
                word = word.toLowerCase(); // Convert the word to lowerCase before searching  
                trie.search(word); // Search the word  
            }  
        }  
    } catch (IOException e) {  
        /* Handle any exceptions that occur while reading the file */  
        System.err.println("Error reading the file: " + e.getMessage());  
    }  
}
```



3. Πρόταση Εναλλακτικών Λέξεων

Η εφαρμογή μας περιλαμβάνει έναν μηχανισμό, ο οποίος προσφέρεται επαναληπτικά, για την αναζήτηση και πρόταση εναλλακτικών λέξεων. **Δέχεται από τον χρήστη μία λέξη και έναν αριθμό k**, ο οποίος καθορίζει το πλήθος των **εναλλακτικών λέξεων που θα εμφανιστούν ως προτάσεις**. Οι λέξεις που προτείνονται πληρούν ένα ή περισσότερα από τα παρακάτω κριτήρια ομοιότητας με τη λέξη εισόδου:

```
Give the prefix word:
so
Give the amount of words you want us to suggest:
3
```

- **Πρόθεμα:** Η λέξη εισόδου είναι πρόθεμα της προτεινόμενης λέξης.

Παράδειγμα δοθείσας λέξης: so. Προτεινόμενες λέξεις: something, soul, souls, sort, solid, solution.

```
Algorithm 1:

something, 11
soul, 2
souls, 2
```

- **Ίδιο μήκος με διαφορά χαρακτήρων:** Η προτεινόμενη λέξη έχει το ίδιο μήκος με τη λέξη εισόδου και διαφέρει από αυτήν το πολύ κατά δύο χαρακτήρες.

Παράδειγμα δοθείσας λέξης: so. Προτεινόμενες λέξεις: to, of, is, no, at.

```
Algorithm 2:

to, 40
of, 34
is, 34
```

- **Διαφορετικό μήκος:** Η προτεινόμενη λέξη έχει μήκος είτε έως και δύο χαρακτήρες μεγαλύτερο από τη λέξη εισόδου, είτε ένα χαρακτήρα λιγότερο. Σε περίπτωση που το μήκος είναι μεγαλύτερο, θα υπάρχει πιθανότητα η λέξη να μην εμφανίζεται αυτούσια στην προτεινόμενη λέξη (επιτρέπεται η εμφάνιση χαρακτήρων ενδιάμεσα των χαρακτήρων της λέξης).

Παράδειγμα δοθείσας λέξης: so. Προτεινόμενες λέξεις: s, also, snow, show, soap, sold.

```
Algorithm 3:

s, 12
also, 3
show, 4
```

Σημείωση: Για την αποθήκευση των εναλλακτικών λέξεων, χρησιμοποιήσαμε έναν σωρό ελαχίστων (min-heap), ο οποίος διατηρεί τις k λέξεις με τη μεγαλύτερη τιμή σημαντικότητας ανάμεσα στις εναλλακτικές προτάσεις.

```
public class MinHeap implements UsefulNumbers {  
  
    private WordNode[] nodes; // Array holding the heap's nodes  
    private int size; // Current size of the heap  
    private int maxSize; // Maximum size of the heap  
  
    /**  
     * The WordNode class represents a node that holds a word and its associated importance.  
     * It is used for managing words in data structures where the importance of each word is tracked.  
     * The class provides constructors to initialize the word and importance, as well as getter and setter methods.  
     * This can be useful in applications such as ranking or sorting words based on importance.  
     *  
     * @since 27/11/2024  
     * @version 1.0  
     * @author Anastasis Zachariou  
     */  
    public class WordNode implements UsefulNumbers {  
  
        private String word; // The word stored in this node  
        private int importance; // The importance value associated with the word  
  
        /**  
         * Combine all three algorithms to find the top words */  
        MinHeap topKWords = Algorithms.creator(k); /* Create MinHeap for the top words */  
  
        /** Apply all algorithms to combine results */  
        topKWords = Algorithms.prefix(trie, topKWords, word);  
        topKWords = Algorithms.sameLength(trie, topKWords, word);  
        topKWords = Algorithms.differentLength(trie, topKWords, word);  
  
        /* Sort words in the MinHeap */  
        MinHeap.heapSort(topKWords.getNodes(), k);  
    }  
}
```

Top words

to, 40
of, 34
is, 34

4. Βελτιστοποίηση και Απόδοση

Και οι τρεις αλγόριθμοι εφαρμόζονται με τη χρήση της αναδρομής!

Πιο συγκεκριμένα στους τρεις αλγόριθμους διασχίζουμε το λεξικό με την αναδρομικά με DFs με απώτερο σκοπό να βρούμε το σύνολο των λέξεων που αποτελούν το λεξικό καθώς και την σημαντικότητα της κάθε λέξης.

Αδιαμφισβήτητα, για τον κάθε αλγόριθμο διατρέχουμε το λεξικό στα σημεία που χρειάζεται!!!

Πιο συγκεκριμένα, στον πρώτο αλγόριθμο αναζητούμε την λέξη εισόδου που αποτελεί το πρόθεμα και έπειτα διατρέχουμε το λεξικό από το συγκεκριμένο σημείο και έπειτα, όπου βρήσκετε ο τελευταίος χαρακτήρας της λέξης!

Για τον δεύτερο αλγόριθμο, εφόσον ψάχνουμε λέξεις ίδιου μεγέθους βρίσκουμε το επίπεδο του λεξικού που αντιστοιχεί στο μέγεθος της λέξης και διατρέχουμε το δέντρο μέχρι το συγκεκριμένο επίπεδο!

Τέλος, για τον τρίτο αλγόριθμο αναγκαστικά διατρέχουμε όλο το λεξικό εφόσον ψάχνουμε λέξεις διαφορετικού βεληνεκούς από το μέγεθος της εισαχθείσας λέξεις!

Μέρος 2ο:

Πειραματική Αξιολόγηση των Δύο Υλοποιήσεων Trie ως προς την Κατανάλωση Μνήμης

Στο δεύτερο μέρος, η εργασία επικεντρώνεται στη συγκριτική ανάλυση των υλοποιήσεων της δομής Trie ως προς την κατανάλωση μνήμης που απαιτείται για την αποθήκευση του λεξικού.

Οι δύο εκδοχές είναι:

- **Υλοποίηση με χρήση στατικού πίνακα:** Σε αντίθεση με την υλοποίηση που έχουμε δει στα πλαίσια του εργαστηρίου, κάθε θέση του πίνακα στον κόμβο του Trie αποτελείται από το στοιχείο το οποίο φυλάσσεται και τον κόμβο του Trie στον οποίο παραπέμπει η συγκεκριμένη θέση. Κάθε κόμβος του Trie περιέχει και τη σημαντικότητα της λέξης που τερματίζεται εκεί. Δηλαδή η θέση του χαρακτήρα προς εισαγωγή δε θα υπολογίζεται πλέον με τη χρήση του πίνακα ASCII.

```
public class StaticTrie implements UsefulNumbers {  
    private StaticTrieNode root; // Root node of the StaticTrie, starts with an empty node  
  
    /* Inner class StaticTrieNode represents a node in the StaticTrie */  
    private class StaticTrieNode {  
  
        /* Array to hold children nodes for each letter of the alphabet */  
        StaticTrieNode[] letters = new StaticTrieNode[ALPHABET_SIZE];  
        int wordLength; // Stores the length of the word if it ends at this node  
  
        /**  
         * Constructor to initialize a StaticTrieNode.  
         * The node starts with no children, and wordLength is zero (meaning no word ends here).  
         *  
         * @since 11/11/2024  
         * @version 1.0  
         * @author Anastasis Zachariou  
         */  
        public StaticTrieNode() {  
  
            this.wordLength = ZERO; // No word ends at this node initially  
  
            /* Initialize the letters array with null values, as no children exist yet */  
            for(int i = ZERO; i < ALPHABET_SIZE; i++) {  
  
                this.letters[i] = null; // Each position corresponds to a letter of the alphabet  
  
            }  
  
        }  
  
    }  
  
    /**  
     * Calculates the memory consumed by the StaticTrie.  
     * The calculation is done by multiplying the number of nodes by the number of pointers and the size of the array.  
     * Each node contains an array of 26 elements (for each letter of the alphabet) and a pointer to each element of the array.  
     * The total memory is calculated as: (number of nodes) * (size of the array) * (size of a pointer).  
     *  
     * @return The total memory in bytes consumed by the StaticTrie.  
     *  
     * @since 30/11/2024  
     * @version 1.0  
     * @author Anastasis Zachariou  
     */  
    public int getMemoryBytes() {  
  
        /* Calculate the number of nodes in the StaticTrie */  
        int nodesNumber = this.getNodesNumber();  
  
        /* Size of the pointer in bytes (4 bytes per pointer) */  
        int bytesPointer = FOUR;  
  
        /* Size of the letters array for each node (26 elements for the alphabet) */  
        int sizeTable = ALPHABET_SIZE;  
  
        /* Calculate the total memory in bytes */  
        int memoryBytes = (nodesNumber * bytesPointer * sizeTable);  
  
        return (memoryBytes); // Return the total memory in bytes  
  
    }  
}
```

26 * 4 Bytes

- **Υλοποίηση με χρήση πίνακα κατακερματισμού τύπου Robin Hood:** Όπως αυτή υλοποιήθηκε στο μέρος 1 της εργασίας.

```

public class RobinHoodTrie implements UsefulNumbers {
    private RobinTrieNode root; // Root node of the RobinHoodTrie, starts with an empty node

    public class RobinTrieNode implements UsefulNumbers {
        /* A reference to the RobinHoodHashing object associated with this TrieNode */
        RobinHoodHashing robinHood;
        int wordLength; // Stores the length of the word if it ends at this node
        int importance; // The importance value associated with the word
    }

    public class RobinHoodHashing implements UsefulNumbers {
        private Element[] table; // The hash table holding the elements
        private int capacity; // The capacity of the table
        private int size; // The number of elements currently in the table
        private int maxProbeLength; // The maximum probe length encountered
    }

    public class Element implements UsefulNumbers {
        private char key; // The character key of the element
        private int probeLength; // The probe length from the initial hash position
        private RobinTrieNode trieNode; // The RobinTrieNode that this Element points to
    }

    /**
     * Calculates the memory usage of the RobinHoodTrie in bytes.
     * This method calculates the total memory usage by considering the size of nodes, elements,
     * and the RobinHood hashing structure.
     *
     * @return The total memory usage in bytes for the RobinHoodTrie.
     *
     * @since 30/11/2024
     * @version 1.0
     * @author Anastasis Zachariou
     */
    public int getMemoryBytes() {
        /* Calculate the number of nodes in the RobinHoodTrie */
        int nodesNumber = this.getNodesNumber();

        /* Get the total number of elements in the trie */
        int elementsNumber = this.getElementsNumber();

        int bytesProbeLength = FOUR; // Size of probe length in bytes
        /* Size of the pointer in bytes (4 bytes per pointer) */
        int bytesPointer = FOUR;

        int bytesElement = (bytesPointer + bytesProbeLength); // Size of each element in the table

        int bytesCapacity = FOUR; // Size of the capacity field in bytes
        int bytesSize = FOUR; // Size of the size field in bytes
        int bytesMaxProbeLength = FOUR; // Size of the max probe length field in bytes

        /* Calculate the memory used by the RobinHood hashing structure */
        int bytesRobinHoodHashing = ((bytesCapacity + bytesSize + bytesMaxProbeLength) * nodesNumber);

        /* Calculate the total memory in bytes */
        int memoryBytes = ((elementsNumber * bytesElement) + (bytesRobinHoodHashing));

        return (memoryBytes); // Return the total memory in bytes
    }
}

```

Diagram illustrating the memory calculation for the RobinHoodTrie structure:

- sizeof(int = 4 Bytes)**: Points to the `int` fields in the `RobinTrieNode` and `RobinHoodHashing` classes.
- sizeof(pointer = 4 Bytes)**: Points to the `RobinTrieNode` field in the `Element` class.

Στο πλαίσιο της εργασίας θα θέλαμε να συγκρίνουμε την μνήμη που καταναλώνει το Static Trie και το Robin Hood Trie. Έτσι, βλέποντας τα δύο λεξικά σαν δύο συναρτήσεις όπου έχουν αρκετά διαφορετικά και ελάχιστα όμοια παιδιά.

Αφαιρώντας τις δύο συναρτήσεις καταλήγουμε σε δύο εντελώς διαφορετικές συναρτήσεις που επηρεάζονται από διαφορετικά παιδιά! (Αυτό επιτυγχάνεται καθώς παρατηρούμε ότι ο **αριθμός των κόμβων των δύο λεξικών έχουν σχεδόν αμελητέα διαφορά και πάντοτε υπερτερείει αριθμών κόμβων το Static Trie**, έτσι προσεγγιστικά παίρνουμε ότι τα όμοια στοιχεία μπορούν αφαιρεθούν και να εξαλειφθούν από τις δύο συναρτήσεις, **καθώς το τελικό αποτέλεσμα και παρατηρήσεις δεν αλλάζουν.**)

Έτσι θα παρατηρήσετε ότι κατά των υπολογισμών της **συνολικής μνήμης που καταναλώνει το κάθε λεξικό δε συμπεριλαμβάνονται οι μεταβλητές importance, wordLength και key**, εφόσον είναι κείνες και για τα δύο λεξικά!

1. Μεταβλητές Πειράματος:

Κατά την πειραματική ανάλυση, παρατηρήτε πώς η κατανάλωση μνήμης επηρεάζεται από τις παρακάτω συνθήκες. Τα λεξικά παράγονται τυχαία, δεν είναι υποχρεωτική η χρήση πραγματικών λεξικών.

- **Πλήθος των λέξεων στο λεξικό (n).** Εξετάστε διαφορετικές τιμές του n, π.χ., 1000, 5000, 10000, 100000, 500000 κτλ.
- **Μήκος των λέξεων:** Όλες οι λέξεις του λεξικού έχουν το ίδιο μήκος.

2. Διαδικασία Πειράματος:

- Για κάθε τιμή του n και για τις διαφορετικές περιπτώσεις μήκους λέξεων, φορτώστε τις λέξεις στα λεξικά και **μετρήστε τη μνήμη που χρησιμοποιείται.**
- **Για κάθε μέγεθος λεξικού, εκτελέστε την εφαρμογή με διαφορετικά σύνολα λέξεων**, λαμβάνοντας τον **μέσο όρο** της μνήμης που χρησιμοποιείται για το **συγκεκριμένο μέγεθος λεξικού.**

3. Παρουσίαση Αποτελεσμάτων:

Παρουσιάστε τα αποτελέσματα υπό μορφή γραφημάτων, όπου ο άξονας x θα αναπαριστά τις διαφορετικές τιμές του n (το μέγεθος του λεξικού) και ο άξονας y τη μέση κατανάλωση μνήμης ανά υλοποίηση. Για κάθε διάγραμμα, γράψτε μία σύντομη ανάλυση (μέχρι 8 γραμμές) των παρατηρήσεών σας.

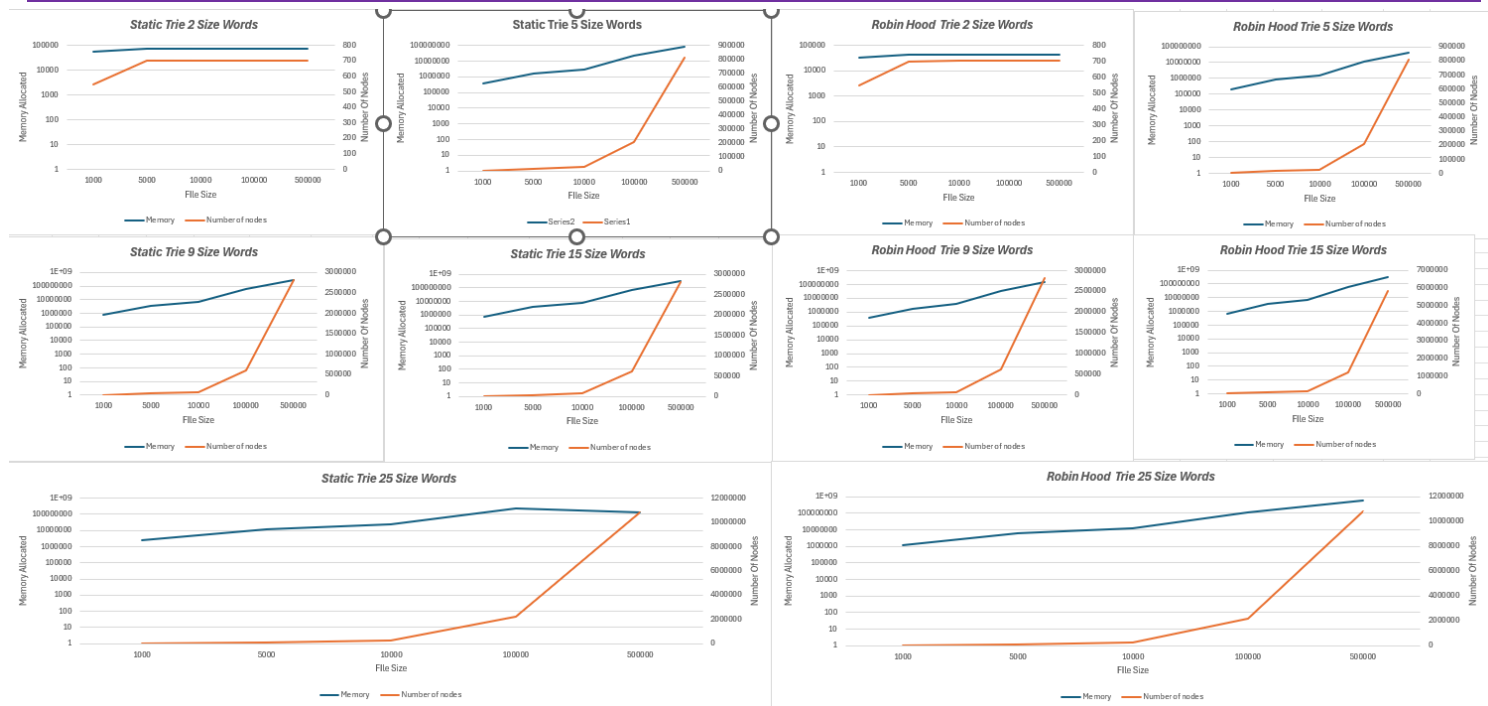
First Run The same length: All the words in the dictionary-trie have the same length.

Static Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	548	56 992
1 000	5	3 516	365 664
1 000	9	7 510	781 040
1 000	15	13 511	1 405 144
1 000	25	23 519	2 445 976
5 000	2	699	72 696
5 000	5	15 008	1 560 832
5 000	9	35 033	3 643 432
5 000	15	65 048	6 764 992
5 000	25	115 010	11 961 040
10 000	2	702	73 008
10 000	5	28 216	2 934 464
10 000	9	68 203	7 093 112
10 000	15	128 290	13 342 160
10 000	25	228 222	23 735 088
100 000	2	702	73 008
100 000	5	207 709	21 601 736
100 000	9	607 609	63 191 336
100 000	15	1 207 426	125 572 304
100 000	25	2 207 726	229 603 504
500 000	2	702	73 008
500 000	5	811 710	84 417 840
500 000	9	2 811 357	292 381 128
500 000	15	5 811 654	604 412 016
500 000	25	10 811 917	1 124 439 368

Robin Hood Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	548	33 440
1 000	5	3 513	188 404
1 000	9	7 510	396 136
1 000	15	13 510	708 120
1 000	25	23 516	1 228 240
5 000	2	699	41 532
5 000	5	15 003	814 652
5 000	9	35 027	1 855 788
5 000	15	65 034	3 417 304
5 000	25	115 009	6 015 572
10 000	2	702	41 688
10 000	5	28 201	1 537 476
10 000	9	68 176	3 616 064
10 000	15	128 258	6 740 264
10 000	25	228 197	11 937 460
100 000	2	702	41 688
100 000	5	207 596	11 460 096
100 000	9	607 466	32 251 704
100 000	15	1 207 298	63 440 856
100 000	25	2 207 595	115 460 508
500 000	2	702	41 688
500 000	5	810 356	45 058 544
500 000	9	2 809 979	149 044 860
500 000	15	5 810 398	305 064 008
500 000	25	10 810 583	565 078 428



Σύμφωνα με τον πιο πάνω πίνακα αλλά και τις πιο πάνω γραφικές παραστάσεις στα δεξιά μας έχουμε τα αποτελέσματα για διαφορετικά μεγέθη αρχείων και διαφορετικά μεγέθη λέξεων για το Robin Hood Trie καθώς και το συνολικό αριθμό των κόμβων και της μνήμης που καταναλώνει το κάθε λεξικό σε κάθε περίπτωση. Ομοίως στα αριστερά για το Static Trie.

Για τα γραφήματα ο άξονας των x αντιπροσωπεύει το μέγεθος του αρχείου, ο άξονας των y στα αριστερά παρίσταται με τις τιμές της μνήμης και ο άξονας των y στα δεξιά παρίσταται και με βάση τις τιμές του αριθμού των κόμβων.

Second Run The same length: All the words in the dictionary-trie have the same length.

Static Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	547	56 888
1 000	5	3 516	365 664
1 000	9	7 524	782 496
1 000	15	13 521	1 406 184
1 000	25	23 532	2 447 328
5 000	2	701	72 904
5 000	5	15 024	1 562 496
5 000	9	35 033	3 643 432
5 000	15	65 023	6 762 392
5 000	25	115 011	11 961 144
10 000	2	702	73 008
10 000	5	28 261	2 939 144
10 000	9	68 233	7 096 232
10 000	15	128 184	13 331 136
10 000	25	228 228	23 735 712
100 000	2	702	73 008
100 000	5	207 650	21 595 600
100 000	9	607 698	63 200 592
100 000	15	1 207 370	125 566 480
100 000	25	2 207 698	229 600 592
500 000	2	702	73 008
500 000	5	812 301	84 479 304
500 000	9	2 811 561	292 402 344
500 000	15	5 811 493	604 395 272
500 000	25	10 811 826	1 124 429 904

Robin Hood Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	547	33 388
1 000	5	3 513	188 244
1 000	9	7 524	396 656
1 000	15	13 518	708 392
1 000	25	23 531	1 229 484
5 000	2	701	41 636
5 000	5	15 015	815 292
5 000	9	35 031	1 856 364
5 000	15	65 022	3 416 664
5 000	25	115 006	6 015 512
10 000	2	702	41 688
10 000	5	28 245	1 540 020
10 000	9	68 210	3 618 040
10 000	15	128 166	6 735 032
10 000	25	228 213	11 937 508
100 000	2	702	41 688
100 000	5	207 540	11 456 256
100 000	9	607 596	32 261 056
100 000	15	1 207 270	63 439 704
100 000	25	2 207 577	115 456 292
500 000	2	702	41 688
500 000	5	810 911	45 092 988
500 000	9	2 810 221	149 058 068
500 000	15	5 810 095	305 051 580
500 000	25	10 810 440	565 071 952

Παρατηρούμε ότι για ίδιο μέγεθος λέξεις, καθώς το μέγεθος του αρχείου αυξάνεται, αναλόγως αυξάνεται και ο αριθμός των κόμβων για κάθε λεξικό, καθώς και ο αριθμός της συνολικής μνήμης που καταναλώνει το κάθε λεξικό. Σύμφωνα με τα δύο λεξικά **παρατηρείται** ότι ο **συνολικός αριθμός των κόμβων για το Static Trie είναι ελάχιστα μεγαλύτερος σε σύγκριση με το συνολικό αριθμό των κόμβων στο λεξικό Robin Hood Trie**. Επίσης, **ανάλογη** σχέση υπάρχει και στην συνολική μνήμη του κάθε λεξικού εφόσον **ο αριθμός των κόμβων επηρεάζει άμεσα το μέγεθος της μνήμης, με μόνη διαφορά ότι η μνήμη που καταναλώνει το Static Trie είναι ΔΙΠΛΑΣΙΑ από την μνήμη που καταναλώνει το Robin Hood Trie**.

Ακολουθως δίνεται ο **μέσος όρος** του συνολικού αριθμού των κόμβων και της συνολικής μνήμης από τις δύο εκτέλεσης του προγράμματος για τα δύο λεξικά!

Average The same length: All the words in the dictionary-trie have the same length.

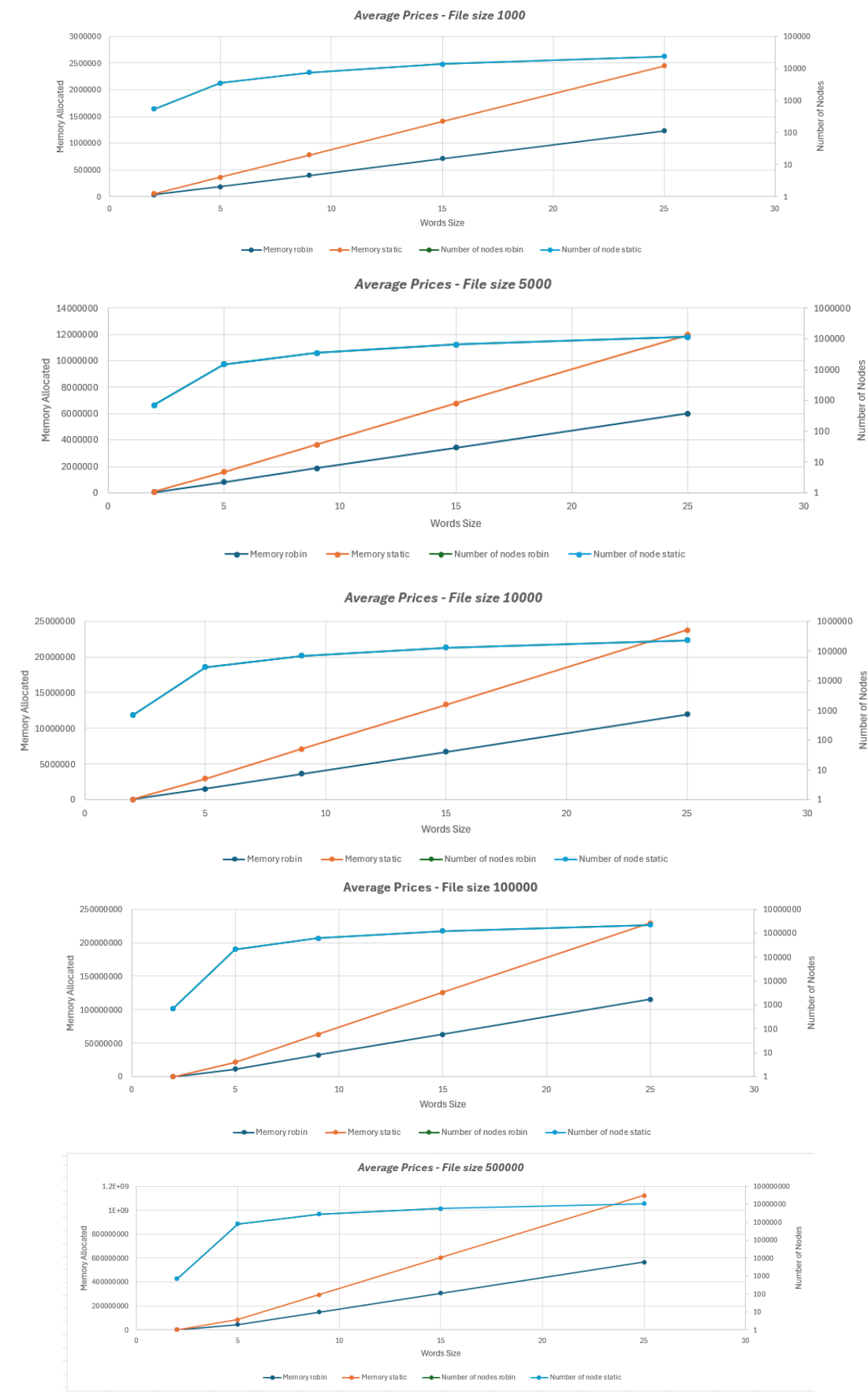
Static Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	547.5	56 940
1 000	5	3 516	365 644
1 000	9	7 517	781 768
1 000	15	13 516	1 405 664
1 000	25	23 525.5	2 446 652
5 000	2	700	72 800
5 000	5	15 016	1 561 664
5 000	9	35 033	3 643 432
5 000	15	65 035.5	6 763 692
5 000	25	115 010.5	11 961 092
10 000	2	702	73 008
10 000	5	28 238.5	2 936 804
10 000	9	68 218	7 094 672
10 000	15	128 237	13 336 648
10 000	25	228 225	23 735 400
100 000	2	702	73 008
100 000	5	207 679.5	21 598 668
100 000	9	607 683.5	63 195 964
100 000	15	1 207 398	125 569 392
100 000	25	2 207 712	229 602 048
500 000	2	702	73 008
500 000	5	812 005.5	84 448 572
500 000	9	2 811 459	292 391 736
500 000	15	5 811 573.5	604 403 644
500 000	25	10 811 872	1 124 434 636

Robin Hood Trie

File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	547.5	33 444
1 000	5	3 513	188 324
1 000	9	7 517	396 396
1 000	15	13 514	708 256
1 000	25	23523.5	1 228 862
5 000	2	700	41 584
5 000	5	15 009	814 972
5 000	9	35 029	1 856 076
5 000	15	65 028	3 416 984
5 000	25	115 007.5	6 015 542
10 000	2	702	41 688
10 000	5	28 223	1 538 748
10 000	9	68 193	3 617 052
10 000	15	128 212	6 737 648
10 000	25	228 205	11 937 484
100 000	2	702	41 688
100 000	5	207 569	11 458 176
100 000	9	607 531	32 256 380
100 000	15	1 207 284	63 440 280
100 000	25	2 207 586	115 456 400
500 000	2	702	41 688
500 000	5	810 633.5	45 075 766
500 000	9	2 810 100	149 051 464
500 000	15	5 810 246.5	305 057 794
500 000	25	10 810 512	565 075 190

Οι παρατηρήσεις της προηγούμενης σελίδας αποδεικνύονται περίτρανα και με τη βοήθεια των πιο κάτω γραφημάτων!



1. Μεταβλητές Πειράματος:

Κατά την πειραματική ανάλυση, παρατηρήτε πώς η κατανάλωση μνήμης επηρεάζεται από τις παρακάτω συνθήκες. Τα λεξικά παράγονται τυχαία, δεν είναι υποχρεωτική η χρήση πραγματικών λεξικών.

- **Πλήθος των λέξεων στο λεξικό (n).** Εξετάστε διαφορετικές τιμές του n, π.χ., 1000, 5000, 10000, 100000, 500000 κτλ.

- **Διάφορα μήκη:** Οι λέξεις του λεξικού ποικίλλουν σε μήκος. Το μήκος των λέξεων δεν πρέπει να είναι παράγεται τυχαία και ομοιόμορφα σε ένα εύρος μήκους (πχ 1 – 20 χαρακτήρες), αλλά θα πρέπει να βασίζεται σε μια κατανομή η οποία να λαμβάνει υπόψιν την κατανομή λέξεων σε ένα πραγματικό λεξικό!

Από που αντλήσατε τις πηγές μας για αυτή την κατανομή!

Με βάση το αρχείο LowerCase_Words και με την βοήθεια της ακόλουθης μεθόδου!

```
public class FileCreator implements UsefulNumbers {  
    /**  
     * Calculates the percentage of words based on their length from a given file.  
     * This method reads the file, counts the occurrence of words of different lengths,  
     * and then calculates the percentage of each word length.  
     *  
     * @param file The file path to the text file containing the words.  
     *  
     * @since 28/11/2024  
     * @version 1.0  
     * @author Neophytos Kaikitis  
     */  
    public static void percentage(String file) {  
        /** Ensure user provided the file path as argument */  
        String filePath = file;  
  
        /** Array to store the count of words for each length */  
        int[] sizes = new int[LARGER_SIZE_WORDS];  
  
        /** Total count of words in the file */  
        int totalWords = ZERO;  
  
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {  
            String line;  
  
            /** Read file line by line */  
            while ((line = br.readLine()) != null) {  
                /** Split the line into words, removing any non-alphabetical characters */  
                String[] words = line.split("[^A-Za-z]+");  
  
                /** Process each word in the line */  
                for (String word : words) {  
                    if (!word.isEmpty()) {  
                        /** Count word length and increment the respective count in the sizes array */  
                        int length = word.length();  
                        sizes[length]++;  
                        totalWords++;  
                    }  
                }  
            }  
        }  
  
        /** Print word length percentages */  
        if (totalWords == ZERO) {  
            System.out.println("No words found in the file.");  
        }  
        else {  
            System.out.println("Word length Percentages:");  
            for (int i = ONE; i < LARGER_SIZE_WORDS; i++) {  
                int length = i;  
                int count = sizes[i];  
                double percentage = (count * 100.0) / totalWords;  
                if (percentage != ZERO) {  
                    /** Display the percentage of each word length */  
                    System.out.printf("Length %d: %f%%\n", length, percentage);  
                }  
            }  
        }  
    }  
    catch (IOException e) {  
        /** Handle potential IOExceptions when reading the file */  
        System.err.println("Error reading the file: " + e.getMessage());  
    }  
}
```

Αποτελέσματα!

```
/**  
 * Returns an array of word lengths that can be used for generating random words. These lengths  
 * represent possible lengths for the randomly generated words, based on statistical analysis or design.  
 *  
 * @return An array of integer word lengths.  
 *  
 * @since 28/11/2024  
 * @version 1.0  
 * @author Neophytos Kaikitis  
 */  
public static int[] getLengths() {  
    int[] lengths = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 27,  
        28, 29, 31, 45 };  
  
    /** Return a predefined set of word lengths */  
    return lengths;  
}  
  
/**  
 * Returns an array of percentages that represent the probability distribution for selecting each  
 * word length. The percentages define the likelihood of choosing each word length from the getLengths() array.  
 *  
 * @return An array of double values representing the percentage distribution for word lengths.  
 *  
 * @since 28/11/2024  
 * @version 1.0  
 * @author Neophytos Kaikitis  
 */  
public static double[] getPercentages() {  
    double[] percentages = { 0.007313, 0.117849, 0.574899, 1.912016, 4.228486, 8.001046, 11.315151, 14.007386,  
        14.273459, 12.311660, 10.161416, 7.911324, 5.726484, 3.904472, 2.447538, 1.449341, 0.835628, 0.415142,  
        0.214040, 0.100652, 0.047533, 0.020532, 0.008436, 0.003375, 0.002250, 0.000844, 0.000563, 0.000563,  
        0.000281, 0.000281 };  
  
    /** Return a predefined array of percentages for word length distribution */  
    return percentages;  
}
```

2. Διαδικασία Πειράματος:

- Για κάθε τιμή του n και για τις διαφορετικές περιπτώσεις μήκους λέξεων, φορτώστε τις λέξεις στα λεξικά και **μετρήστε τη μνήμη που χρησιμοποιείται.**

- **Για κάθε μέγεθος λεξικού, εκτελέστε την εφαρμογή με διαφορετικά σύνολα λέξεων,** λαμβάνοντας τον **μέσο όρο** της μνήμης που χρησιμοποιείται για το **συγκεκριμένο μέγεθος λεξικού.**

3. Παρουσίαση Αποτελεσμάτων:

Παρουσιάστε τα αποτελέσματα υπό μορφή γραφημάτων, όπου ο άξονας x θα αναπαριστά τις διαφορετικές τιμές του n (το μέγεθος του λεξικού) και ο άξονας y τη μέση κατανάλωση μνήμης ανά υλοποίηση. Για κάθε διάγραμμα, γράψτε μία σύντομη ανάλυση (μέχρι 8 γραμμές) των παρατηρήσεών σας.

First Run Percentage

Static Trie

File Size	Number of Nodes	Memory allocated
1 000	7 888	820 352
5 000	37 054	3 853 616
10 000	72 632	7 553 728
100 000	655 282	68 149 328
500 000	3 054 122	317 628 688

Robin Hood Trie

File Size	Number of Nodes	Memory allocated
1 000	7 884	415 376
5 000	37 046	1 960 680
10 000	72 599	3 846 876
100 000	655 183	34 725 900
500 000	3 052 844	161 640 768

Second Run Percentage

Static Trie

File Size	Number of Nodes	Memory allocated
1 000	8 127	845 208
5 000	37 734	3 924 336
10 000	72 945	7 586 280
100 000	657 388	68 368 352
500 000	3 049 617	317 160 168

Robin Hood Trie

File Size	Number of Nodes	Memory allocated
1 000	8 127	428 396
5 000	37 727	1 997 500
10 000	72 931	3 864 236
100 000	657 256	34 835 408
500 000	3 048 425	161 416 996

Average Percentage

Static Trie

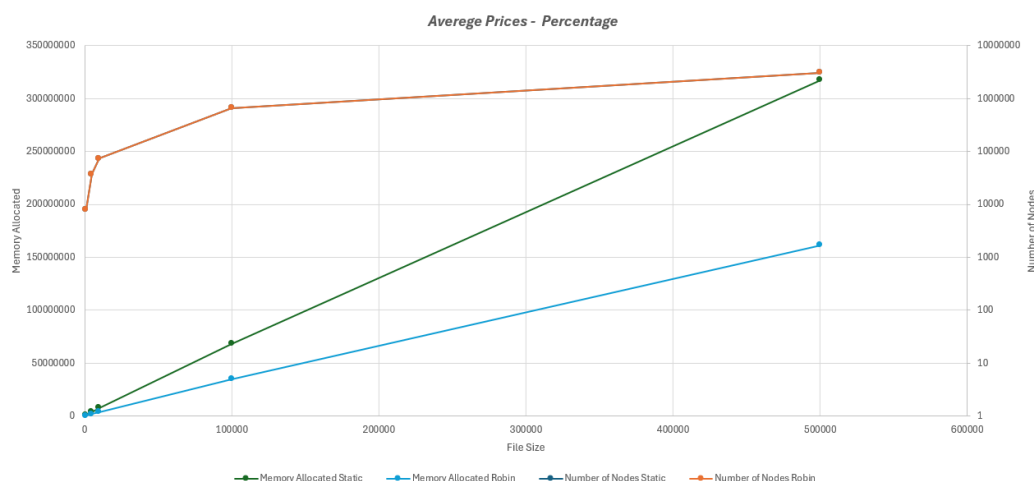
File Size	Number of Nodes	Memory allocated
1 000	8 007.5	832 780
5 000	37 394	3 888 976
10 000	72 789	7 570 004
100 000	656 335	68 258 840
500 000	3 051 869.5	317 394 428

Robin Hood Trie

File Size	Number of Nodes	Memory allocated
1 000	8 005.5	421 886
5 000	37 386.5	1 979 090
10 000	72 765	3 855 556
100 000	656 219.5	34 780 654
500 000	3 050 634.5	161 538 882

Όμοιες παρατηρήσεις έχουμε και σε αυτήν την εφαρμογή λεξικού όπου διαθέτει στο δυναμικό του **διαφορετικού βεληνεκούς λέξεις**.

Πιο συγκεκριμένα ότι **ο αριθμός των κόμβων των δύο λεξικών είναι πανομοιότυπος** και αυτό φαίνεται και στο **πιο κάτω διάγραμμα** όπου τα δύο γραφήματα τον **Number of Nodes** και για τα δύο λεξικά, **ΕΠΙΚΑΛΥΠΤΕΙ το ένα το άλλο**. Καθώς και ότι, η **συνολική μνήμη** που καταναλώνει ο **Static Trie είναι ΔΙΠΛΑΣΙΑ** από αυτήν του **Robin Hood Trie!**



Σύγκριση δύο περιπτώσεων ίδιο μήκος λέξη διαφορετικού μήκους λέξεις

Average The same length: All the words in the dictionary-trie have the same length.

Static Trie				Robin Hood Trie			
File Size	Size Words	Number of Nodes	Memory allocated	File Size	Size Words	Number of Nodes	Memory allocated
1 000	2	547.5	56 940	1 000	2	547.5	33 414
1 000	5	3 516	365 644	1 000	5	3 513	188 324
1 000	9	7 517	781 768	1 000	9	7 517	396 396
1 000	15	13 516	1 405 664	1 000	15	13 514	708 256
1 000	25	23 525.5	2 446 652	1 000	25	23 523.5	1 228 862
5 000	2	700	72 800	5 000	2	700	41 584
5 000	5	15 016	1 561 664	5 000	5	15 009	814 972
5 000	9	35 033	3 643 432	5 000	9	35 029	1 856 076
5 000	15	65 035.5	6 763 692	5 000	15	65 028	3 416 984
5 000	25	115 010.5	11 961 092	5 000	25	115 007.5	6 015 542
10 000	2	702	73 008	10 000	2	702	41 688
10 000	5	28 238.5	2 936 804	10 000	5	28 223	1 538 748
10 000	9	68 218	7 094 672	10 000	9	68 193	3 617 052
10 000	15	128 237	13 336 648	10 000	15	128 212	6 737 648
10 000	25	228 225	23 735 400	10 000	25	228 205	11 937 484
100 000	2	702	73 008	100 000	2	702	41 688
100 000	5	207 679.5	21 598 668	100 000	5	207 568	11 458 176
100 000	9	607 653.5	63 195 964	100 000	9	607 531	32 256 380
100 000	15	1 207 398	125 569 392	100 000	15	1 207 284	63 440 280
100 000	25	2 207 712	229 602 048	100 000	25	2 207 586	115 458 400
500 000	2	702	73 008	500 000	2	702	41 688
500 000	5	812 005.5	84 448 572	500 000	5	810 633.5	45 075 766
500 000	9	2 811 459	282 391 736	500 000	9	2 810 100	149 051 464
500 000	15	5 811 573.5	604 403 644	500 000	15	5 810 246.5	305 057 794
500 000	25	10 811 872	1 124 434 636	500 000	25	10 810 512	565 075 190

Average Percentage

Static Trie			Robin Hood Trie		
File Size	Number of Nodes	Memory allocated	File Size	Number of Nodes	Memory allocated
1 000	8 007.5	832 780	1 000	8 005.5	421 886
5 000	37 394	3 888 976	5 000	37 386.5	1 979 090
10 000	72 789	7 570 004	10 000	72 765	3 855 556
100 000	656 335	68 258 840	100 000	656 219.5	34 780 654
500 000	3 051 869.5	317 394 428	500 000	3 050 634.5	161 538 882

Συγκρίνοντας για του ίδιου βεληνεκούς λέξεις (πρώτος πίνακας) τον συνολικό αριθμό των κόμβων και την συνολική μνήμη που χρειάζονται και τα δύο λεξικά παρατηρούμε ότι είναι **κατά πουλί μεγαλύτερη σχεδόν τριπλάσια οι αριθμοί** σε σχέση με τα λεξικά που απαρτίζονται από διαφορετικού μήκους λέξεις (δεύτερος πίνακας)

Η σύγκριση γίνεται με βάση το εκάστοτε μέγεθος του αρχείου!

Τελικό πόρισμα

Ως προς την μνήμη: Αδιαμφισβήτητα, σύμφωνα με τις πιο πάνω παρατηρήσεις του πιο πάνω πειράματος η υλοποίηση λεξικού με Robin Hood Hashing είναι πιο αποδοτική όσον αφορά την μνήμη σε σχέση με την υλοποίηση του λεξικού με Static Table.

Ως προς την πολυπλοκότητα της δομής: Αναμφίβολα, συγκρίνοντας τις δύο δομές, η πολυπλοκότητα του λεξικού με Robin Hood Hashing είναι μεγαλύτερη εφόσον διαχειρίζεται διαφορετικά αντικείμενα, περισσότερες μεταβλητές και αλγορίθμους σε σχέση με το λεξικό με Static Table όπου διαχειρίζεται απλά έναν στατικό πίνακα.

Ως προς το χρόνο αναζήτησης - διάσχισης: Αποδεικνύεται ότι το κόστος αναζήτησης - διάσχισης χειρότερης περίπτωσης για το Robin Hood Trie είναι φραγμένο από $\Theta(\log \log n)$.